



香港中文大學 (深圳)

The Chinese University of Hong Kong

CSC 5003 Algorithm Art and Programming Practice Lecture 11: Binary Series

Jingbang Chen

School of Data Science (SDS)

The Chinese University of Hong Kong, Shenzhen



Things that we have learned called "Binary"

- ▶ Binary State - State Compression DP
- ▶ Binary Tree - Lecture 3
- ▶ Binary Search
- ▶ Binary Lifting - LCA
- ▶ Binary Indexing (RMQ)

.....

Let's do some review first!
How to code them?



Split Array Largest Sum

- ▶ You are given an integer array `nums` and an integer `k`.
- ▶ Split `nums` into exactly `k` non-empty contiguous subarrays.
- ▶ Your goal is to minimize the largest sum among these `k` subarrays.
- ▶ Return the minimum possible value of that largest subarray sum.



Solution

Key Idea

- ▶ We want to minimize the largest subarray sum after splitting the array into exactly k non-empty contiguous parts.
- ▶ Instead of directly searching for the optimal partition, we ask:
 - If the maximum allowed subarray sum is X , can we split the array into at most k subarrays so that each subarray has sum at most X ?
- ▶ If we can answer this question efficiently, then we can binary search for the smallest feasible X .



Solution

- ▶ **Why Binary Search Works**
- ▶ Let $\text{check}(X)$ mean:
 - it is possible to split the array into at most k subarrays such that every subarray sum is at most X .
- ▶ Then:
 - if $\text{check}(X)$ is true, then $\text{check}(Y)$ is also true for any $Y > X$
 - if $\text{check}(X)$ is false, then $\text{check}(Y)$ is also false for any $Y < X$
- ▶ So the predicate is **monotone**, which makes binary search applicable.



Solution

- ▶ At minimum, the answer must be at least the largest single element, because every element must belong to some subarray: $\text{left} = \max(\text{nums})$
- ▶ At maximum, we can put everything into one subarray: $\text{right} = \text{sum}(\text{nums})$



Solution

- ▶ Now we need to implement $\text{check}(X)$.
- ▶ We scan the array from left to right and greedily form subarrays:
- ▶ keep adding numbers to the current subarray while its sum does not exceed X
- ▶ if adding the next number would exceed X , start a new subarray
- ▶ This produces the **minimum number of subarrays** needed under the limit X .
- ▶ If the number of subarrays needed is at most k , then X is feasible.



Solution

Why the Greedy Check Is Correct

- ▶ To minimize the number of subarrays under a fixed upper bound X , we should make each subarray as long as possible.
- ▶ Suppose we stop a subarray earlier even though we could still add the next element without exceeding X . Then we only create more subarrays later, which can never help.
- ▶ So the greedy strategy gives the minimum possible number of subarrays for that X .
- ▶ Thus:
 - if greedy uses more than k subarrays, then no valid split exists
 - if greedy uses at most k , then a valid split exists



Kth Smallest Product of Two Sorted Arrays

- ▶ You are given two **sorted** integer arrays `nums1` and `nums2`, and a positive integer `k`.
- ▶ Consider all products of the form:
 - $\text{nums1}[i] * \text{nums2}[j]$ for every valid pair (i, j) .
- ▶ Return the **k-th smallest** product among all these products.



Solution

- ▶ We do **not** try to generate all products, because there can be up to: $\text{nums1.length} * \text{nums2.length}$, which is far too large.
- ▶ Instead, we binary search a value x and ask:
 - How many pairs (i, j) satisfy $\text{nums1}[i] * \text{nums2}[j] \leq x$?
- ▶ If we can compute this count efficiently, then we can binary search for the smallest x such that the count is at least k .
- ▶ That x is exactly the k -th smallest product.



Solution

Why Binary Search Works

- ▶ Define:
 - $\text{count}(x)$ = number of pairs (i, j) such that $\text{nums1}[i] * \text{nums2}[j] \leq x$
- ▶ As x increases, $\text{count}(x)$ never decreases.
- ▶ So $\text{count}(x)$ is monotone, and we can binary search for the smallest x with: $\text{count}(x) \geq k$



Solution

- ▶ The product values can be negative or positive, so we choose a wide enough range: left = $-1e10$, right = $1e10$.
- ▶ This is safe because:
 - each number is at most 10^5 in absolute value,
 - so each product is at most 10^{10} in absolute value.



Solution

- ▶ For each $a = \text{nums1}[i]$, we count how many b in nums2 satisfy: $a * b \leq x$
- ▶ The method depends on the sign of a .

Case 1: $a > 0$

- ▶ Then dividing preserves the inequality: $b \leq x / a$
- ▶ So we want the number of elements in nums2 less than or equal to x / a .
- ▶ Since nums2 is sorted, we can use binary search.
- ▶ In integer arithmetic, to avoid precision issues, we simply binary search directly on the condition: $a * \text{nums2}[\text{mid}] \leq x$ and find the last valid position.



Solution

Case 2: $a < 0$

- ▶ Now dividing flips the inequality: $b \geq x / a$
- ▶ So we want the number of elements in `nums2` greater than or equal to some threshold.
- ▶ Again, because `nums2` is sorted, we can binary search for the first valid position satisfying: $a * \text{nums2}[\text{mid}] \leq x$ and count the suffix.

Case 3: $a == 0$

Then: $a * b = 0$

So if $x \geq 0$, then all `nums2.size()` pairs are valid; otherwise, none are valid.



Minimum Edge Weight Equilibrium Queries in a Tree

- ▶ You are given an undirected tree with n nodes numbered from 0 to $n - 1$.
- ▶ The tree is described by $n - 1$ edges. Each edge is given as:
 - $[u, v, w]$ meaning there is an edge between nodes u and v with weight w .
- ▶ You are also given a list of queries, where each query is:
 - $[a, b]$ For each query, consider the unique path from node a to node b .
 - In one operation, you may change the weight of any one edge on this path to any value you want.
 - Return an array `answer`, where `answer[i]` is the minimum number of operations needed so that **all edge weights on the path in the i -th query become equal**.



Solution

- ▶ For a query (u, v) , look at all edge weights on the path from u to v .
- ▶ Suppose the path contains: len total edges
- ▶ among them, the most frequent weight appears mx times
- ▶ Then the minimum number of changes needed is: $\text{len} - \text{mx}$
- ▶ because the best strategy is to keep the most frequent weight and change every other edge to that value.
- ▶ So the real problem is:
 - for each query path, find the number of edges on the path
 - the frequency of each weight on the path
- ▶ Using binary lifting to compute LCA, and the frequency for each character separately.



Rorororobot

- ▶ There is a grid with n rows and m columns. Rows are numbered from 1 to n from **bottom to top**, and columns are numbered from 1 to m from **left to right**. In column i , the bottom $a[i]$ cells are blocked, that is, rows 1, 2, ..., $a[i]$ are blocked and the remaining cells are free.
- ▶ A robot moves in this grid. You may send it commands:
 - move up
 - move down
 - move left
 - move right
- ▶ However, the robot is broken: every command you send is executed exactly k times, so one command moves the robot by exactly k cells in that direction. If the robot ever tries to leave the grid or enter a blocked cell, it explodes. You cannot interrupt a command while it is being executed.
- ▶ For each query, you are given:
 - a start cell (x_s, y_s)
 - a finish cell (x_f, y_f)
 - a value k
- ▶ Determine whether it is possible to send some sequence of commands, possibly empty, so that the robot starts at the start cell and ends **exactly** at the finish cell. Reaching the finish cell in the middle of a command does not count; it must stop there after finishing all commands.



Solution

Observation 1: Divisibility Conditions

- ▶ Since every vertical move changes the row by exactly k , to get from x_s to x_f we must have:
 - $\text{abs}(x_s - x_f) \% k == 0$
- ▶ Similarly, every horizontal move changes the column by exactly k , so we also need:
 - $\text{abs}(y_s - y_f) \% k == 0$
- ▶ If either condition fails, the answer is immediately "NO".



Solution

Observation 2: We Want to Move as High as Possible

- ▶ Suppose we want to move horizontally across columns between y_s and y_f . In column j , all rows $1..a[j]$ are blocked, so to stand safely in that column we need: $\text{row} > a[j]$
- ▶ Therefore, if the maximum blocked height among columns in $[\min(y_s, y_f), \max(y_s, y_f)]$ is mx , then any horizontal traversal must happen on some row strictly greater than mx .
- ▶ Because vertical moves also happen in jumps of size k , from row x_s the rows we can reach are exactly: $x_s, x_s + k, x_s + 2k, \dots$ up to at most n . So the best row we can reach is the highest row top such that: $top \leq n$ and $(top - x_s) \% k == 0$
- ▶ That row is: $top = x_s + ((n - x_s) / k) * k$



Solution

- ▶ If even this highest reachable row is not above mx , then we can never get high enough to pass over the blocked columns. So we need: $top > mx$. This is the core geometric condition.
- ▶ Each query needs: $\max(a[l..r])$, where: $l = \min(ys, yf)$, $r = \max(ys, yf)$. Precompute a RMQ sparse table to solve it.



Meteor

- ▶ There are N countries and M sectors arranged on a circle, numbered from 1 to M .
- ▶ Each sector belongs to exactly one country. For country i , you are also given a target amount $\text{need}[i]$.
- ▶ Then K meteor showers will happen in order.
The t -th meteor shower is described by three integers: $l[t]$, $r[t]$, $\text{add}[t]$ meaning that every sector in the circular interval from $l[t]$ to $r[t]$ receives $\text{add}[t]$ resources.
 - If $l[t] \leq r[t]$, this is the usual interval $[l[t], r[t]]$.
 - If $l[t] > r[t]$, then the interval wraps around the circle, so it affects: $[l[t], M]$ and $[1, r[t]]$.
- ▶ A country gains all resources that fall on the sectors it owns.
- ▶ For each country, determine the earliest meteor shower after which its accumulated resources are at least its target.



Why this problem is hard?

- ▶ A single country is easy:
 - binary search the first shower where it reaches its target;
 - Simulate the first mid showers with a data structure; check how much total resource lands on that country's sectors.
 - But doing this independently for all N countries is too slow.
- ▶ However, we cannot do a binary search for every country.
- ▶ Let's try doing all binary searches together!!!
→ Namely, the parallel binary search trick!



Solution

- ▶ For each country i , define its answer as:
 - the smallest shower index t such that after applying showers $1..t$, the total resources on its sectors are at least $\text{need}[i]$.
- ▶ This answer is monotone:
 - if country i is satisfied after shower t , then it is also satisfied after any later shower.
- ▶ So every country has its own binary search answer.



Solution

- ▶ The problem is that doing N separate binary searches repeats the same meteor-shower simulation many times.
- ▶ Parallel binary search avoids that:
 - we binary search all countries' answers **together**;
 - during one recursion level, we apply one prefix of showers once;
- ▶ then we split countries into:
 - those already satisfied,
 - those not yet satisfied.



Solution

- ▶ We need to apply a shower quickly.
- ▶ A shower adds x to every sector in a circular interval.
- ▶ A convenient Fenwick tree usage is:
 - range add
 - point query
- ▶ For a normal interval $[l, r]$:
 - $\text{diff}[l] += x$
 - $\text{diff}[r+1] -= x$
- ▶ For a wrapped interval: $[l, M]$ and $[1, r]$. we split it into two updates.
- ▶ When querying a sector gives the total meteor amount that has fallen on that sector so far.



Recursive Parallel Binary Search

We solve answers in a half-open range of shower indices:

$[L, R)$

Interpretation:

- every country currently under consideration has its answer somewhere in $[L, R)$

Base case:

- if $L + 1 == R$, then all those countries have answer L

Recursive step:

- Let $mid = (L + R) / 2$
- Apply all showers in $[L, mid)$ to the BIT
- For each country in this group:
 - compute total resources currently obtained on its sectors
 - if this is enough, it belongs to the **left half**
 - otherwise:
 - subtract that gained amount from its remaining target
 - it belongs to the **right half**
- Roll back the showers in $[L, mid)$
- Recurse on both halves